Efficient Finger Model and Accurate Tracking for Hover-and-Touch, Mid-air and Microgesture Interaction

Quentin Zoppis
Univ. Grenoble Alpes, CNRS, Inria, Grenoble INP, LIG, LJK
Grenoble, France
quentin.zoppis@univ-grenoble-alpes.fr

Sergi Pujades
Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK
Grenoble, France
sergi.pujades-rocamora@inria.fr

Laurence Nigay
Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG
Grenoble, France
laurence.nigay@univ-grenoble-alpes.fr

François Bérard
Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG
Grenoble, France
francois.berard@univ-grenoble-alpes.fr

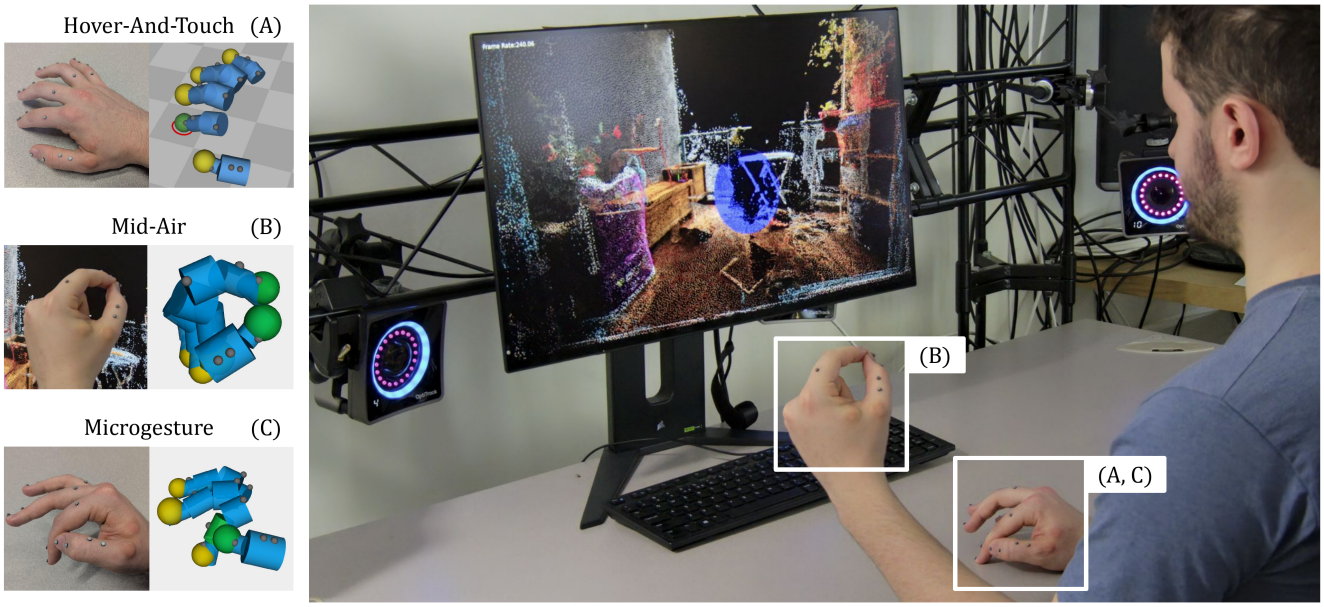

Figure 1: Marker-based visual tracking of fingers using a simple model made of cylinders and a contact sphere. Touch events are detected from the distance between a fingertip modelled as a sphere and a planar touch surface (A), another fingertip (B) or a phalanx modelled as a cylinder (C). We illustrate the approach with a demonstration application combining hover-and-touch, mid-air and microgesture interactions for the edition of a 3D point cloud.

Abstract

Bare-handed gestural interaction with computer systems is widespread, whether with touchscreens or Augmented Reality headsets. Various forms of gestural interaction exist including hover-and-touch, mid-air and microgesture interaction. Studying the full benefits of these gestural interactions, and their combinations, is currently not possible due to the inadequate performances of the existing tracking solutions. To address this problem, we propose a marker-based visual tracking algorithm with a novel finger model, and its open source implementation[1]. A key contribution is the simplicity of the finger and fingertip model (i.e. cylinders and a sphere respectively). This simple model leads to low computational cost (~600 μs), high precision (0.02 mm) and accurate (one millimeter) fingertip tracking, without impeding finger movement. We illustrate the benefits of the proposed tracking approach with a demonstration application combining hover-and-touch, mid-air and microgesture interactions for editing a 3D point cloud.

UIST '25, Busan, Republic of Korea

ACM ISBN 979-8-4007-2037-6/25/09
https://doi.org/10.1145/3746059.3747701

[1] https://iihm.imag.fr/zoppis/maft

CCS Concepts

• **Human-centered computing → Human computer interaction (HCI)**; • **Computer systems organization → Real-time systems**.

Keywords

Hand/Finger Tracking, Articulated Finger Model, Hover-and-Touch, Mid-air, Microgesture, Touch Accuracy, Latency.

ACM Reference Format:
Quentin Zoppis, Sergi Pujades, Laurence Nigay, and François Bérard. 2025. Efficient Finger Model and Accurate Tracking for Hover-and-Touch, Mid-air and Microgesture Interaction. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25), September 28–October 01, 2025, Busan, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3746059.3747701

1 Introduction

Bare-handed gestural interaction with computer systems is widespread, whether with touchscreens or Augmented Reality headsets. This gestural interaction takes several forms. These include hover-and-touch interaction on flat surfaces [3, 72], mid-air interaction with large gestures in the air [27, 68], and microgesture interaction involving rapid and subtle finger movements (one finger in relation to another finger of the same hand) [9].

These gestural interactions are based (i) on the tracking of finger movements, regardless of whether they are in contact with a surface, and (ii) on the detection of touch. Several studies have been carried out on finger tracking and touch detection. Some tracking solutions are defined to have adequate performance when they allow interactions to be studied without adding a negative bias due to tracking performance. However, these tracking solutions are limited to certain forms of gesture interaction. For example, capacitive sensing is a very effective technology for implementing tactile interaction [41], but it does not allow fingers to be tracked when they are not touching the surface.

In this context, our goal is to propose a finger tracking solution for gesture interaction that minimizes as much as possible the impact of the tracking system's imperfections on gesture interaction, i.e., exhibiting adequate performance. This tracking solution thus defines a research platform for studying all the potential advantages of these gestural interactions and their combinations.

To this end, the paper introduces a marker-based visual tracking approach with a novel finger model. We model fingers by two or three cylinders and a *contact sphere* at the fingertip. Four 3 mm markers are attached to each finger. We calibrate a finger model to each specific *shape* of each finger. During online tracking, we optimize the *pose* of the models (i.e., joint angles and position). This allows accurate positioning of the contact sphere. We perform pose estimation via a fast gradient-descent-based algorithm whose execution time does not exceed 600 μs. The simplicity of the model is key to the speed of the approach, while concomitantly ensuring one millimeter accuracy for touch detection and touch position. These performances open the way to straightforward implementations of various high-fidelity bare-hand interactions such as touch, mid-air, pressure, discrete and continuous microgestures, and their combinations.

We demonstrate the speed, accuracy and generality of our approach by implementing a demonstration application involving a combination of hover-and-touch, mid-air, and microgesture interactions. The primary contributions described in this paper are:

- a model of the fingers that is specifically optimized for accurate and fast touch detection;
- a calibration procedure to adjust the model to users' fingers;
- a low-cost (~600 μs) online fitting approach;
- an evaluation of the implementation of the approach with 24.9 ms end-to-end latency and one millimeter accuracy of fingertip tracking and touch detection;
- a demonstration application combining hover-and-touch, mid-air, and microgesture interactions.

2 Related Work

We first review the literature on the requirements for *adequate performance* of any tracking system when studying bare-hand interactions. We then review existing tracking systems that equip users and/or the environment with specific sensors, and the computer vision literature requiring minimal or no user equipment.

2.1 Adequate Performance

The main objective of our work is to experiment with various forms of novel bare-hand interactions while reducing, as much as possible, the negative effects of inadequate system performance.

A system's performance parameter is considered *inadequate* when its improvement would lead to a superior user experience of the interaction (e.g., better user task performance or less fatigue). For example, latency is well known for its strong detrimental effect on users' pointing performance [29, 43, 66]. However, there is a limit beyond which improving the system's latency does not yield better user performance: we consider this limit to represent the *adequate level* of system latency. An adequate level of performance depends on the task and interaction. For example, no performance improvement was observed with *indirect* mouse interaction when reducing latency from 25 to 8 ms [43]; but in *direct* touch pointing, users' performance improved from 25 to 10 ms [29]. Besides latency, other performance parameters were studied for their negative effects on interaction, such as display pixel density [81], frame rate [26], temporal jitter [52], and spatial jitter [53, 66], which is strongly related to the system's precision.

When evaluating a novel interaction technique with a system that has inadequate performance, the study's outcome only reveals the qualities of the interaction *in one particular implementation*. However, if the experimental system has adequate performance, it can be assumed that the study's outcome reflects the more fundamental qualities of the interaction. In other words, the study's outcome is more general as it should replicate with any implementation having adequate performance. In addition, when the system's performance is inadequate, it is difficult to distinguish in the study's outcome between what can be explained by the imperfections of the system and what can be explained by the interaction technique. This problem is compounded when comparative studies are carried out. For example, a recent study found that VR controllers and pens improved users' precision by nearly 30% compared to using bare hands [11]. However, the study involved marker-based tracking for

the VR controllers and pens, but markerless tracking for the fingers with an Oculus Quest 2. The tracking system was thus notably more accurate for the controller and pen than for the fingers: Oculus Quest 2 positional error was recently measured at 11 mm [1], an order of magnitude less accurate than the sub-millimeter accuracy typically achieved by marker-based optical tracking. Hence, the study mainly questions bare-hand interaction *when using the Quest 2*; but the more fundamental difference between using a tool vs. bare hands remains an open question. Similarly, Kim et al. acknowledge that their participants' typing performance was constrained by inadequate tracking latency and precision [34], and question "the level of tracking precision required to reproduce smartphone-level typing performance in practice".

We thus need to identify which system performance parameters may be inadequate (i.e., having a detrimental effect on the interaction) and to quantify the adequate level for each of these parameters. We identified four critical performance parameters: latency, accuracy, precision, and unobtrusiveness. As the versatility of our tracking system is also a main objective, for each parameter we consider the most stringent adequate performance level.

2.1.1 End-to-end Latency. The system's end-to-end latency, or latency in short, has long been identified as a major hindrance to user task performance. MacKenzie and Ware observed pointing performance degradation above 25 ms of latency when using a regular computer mouse [43]. However, adequate latency depends on the interaction and the task. For instance, direct interaction, such as direct touch, is more sensitive to latency than indirect interaction. Jota et al. observed statistically significant user performance improvements when going from 25 ms to 10 ms of latency in direct touch dragging and suggest that adequate latency may be lower than 1 ms [29]. Ng et al. observed that latency was *perceived* at notably different levels depending on the task when using the same digital pen implementation [47]: the median level of perceivable latency was 2 ms when dragging a small box, 6 ms when dragging a larger box, and 40 ms when scribbling inside some boundaries.

Overall, the literature indicates that the adequate level of latency varies depending on the interaction and the task, and that direct interaction, such as mid-air tapping on virtual targets in AR or VR, has more stringent requirements than indirect interactions. Previous studies indicate that latency levels above 25 ms for indirect interaction and 10 ms for direct interaction are likely to induce user performance degradation.

2.1.2 Accuracy. We identify that the most stringent requirement on tracking accuracy comes from touch detection. We have come to expect that the slightest contact on a surface is detected, thanks to the adequate touch detection performance of capacitive surfaces. In addition, accurate touch detection is required for efficient target acquisition: with a fast finger hovering toward the target, an early contact detection (i.e., before the finger contacts the surface) results in a large undershoot. And a late contact detection (i.e., requiring more than the slightest touch) results either in a miss-registration, or in forcing users to press more than they would do naturally, slowing them down. With the optical tracking of fingers, touch detection can be computed by measuring a null distance between the finger surface and the touch surface (which can be another finger). Compared to equipping the finger and/or surface with sensors, optical tracking has the benefit of unobtrusiveness and versatility, as long as the touch surface is also tracked. Yet, the requirement on the tracking accuracy is demanding: in a similar context, Bérard reports that "the required accuracy is probably in the sub-millimeter range, as (…) a 1 mm variability in height measurements resulted in some misdetections and false activations" [3].

2.1.3 Precision. The sub-millimeter accuracy requirements for touch detection imply at least sub-millimeter precision. With lower precision, position jitter would generate instabilities at the touch threshold resulting in spurious touch and release events. Besides touch detection, Pavlovych and Stuerzlinger observed that error rates dramatically increased when jitter reached roughly half the size of the target in a 2D pointing task using a computer mouse [53]. This suggests that spatial jitter should be lower than half the size of the smallest target acquirable in a given interaction. With direct touch, the fat finger problem limits the smallest size acquirable in direct touch to around 5 mm [25]. Indirect finger interaction is not affected by the fat finger problem; it is the human limits of finger precision that define the smallest target size that can be comfortably acquired. Previous studies indicate that this limit is around 5 mm when pointing in free space [5], but targets as small as 0.2 mm could be acquired in a reasonable amount of time when the finger was doing small rotations while touching a surface [4]. Overall, the literature indicates that adequate precision is also in the sub-millimeter range.

2.1.4 Unobtrusiveness. The last performance parameter that we consider is unobtrusiveness. Recent efforts in the microgesture and XR research community have explored various ways to equip the arm, hand, or fingers with sensors to detect finger contact and gestures [69, 80]. However, when instrumenting users, researchers must consider the risk of introducing intrusiveness that changes users' behavior. For example, the instrumentation may be cumbersome, or it may seem fragile and entice users into slow motions. An ideal instrumentation should quickly be forgotten by participants as they use it.

2.2 Detecting Touch with Worn Sensors

For gestures requiring only the tracking of hand-to-hand contacts (e.g., microgestures) or hand-to-surface contacts, various types of sensors have been tested. Researchers have explored gloves worn by users with sewn markers [16, 56], sewn capacitive [71] or force [40] sensors, attached capacitive sensors [34], capacitive tattoos on the skin [30, 76], capacitive [37] or magnetic fingernails [10], radio frequency sensors [80] or accelerometers [54] on a wristband, electric rings [12, 31], and piezo-electric sensors attached to the back of the palm [36]. These approaches may offer adequate latency, accuracy, and/or precision for touch detection, but overall performance depends on the other means required for fingertip positional tracking. In addition, there is a high risk that the worn equipment is intrusive and prevents the observation of ecological user behavior.

2.3 Detecting Touch with Vision Systems

Vision systems have high potential for adequate unobtrusiveness and rich versatility. However, while some provide adequate latency, most still lack adequate accuracy.

We start this section with a discussion on how to evaluate touch detection with respect to our adequate performance parameters. Then, as most vision approaches rely on hand models, we review them before discussing existing approaches to detect touch with vision systems.

2.3.1 Evaluation of Touch Detection. A fundamental question is how to evaluate the accuracy and precision of touch detection. Many works evaluate touch accuracy as a classification task [17–19, 59, 60, 74, 78], i.e., what is the number of correctly classified touch versus non-touch instances. But the classification metric has an important shortcoming: it is highly task-dependent. The same tracking system evaluated on a slow touching task and a fast one may have different results because users will not perform the same movements, applying different speeds and pressures. A good example is [19] reporting 96.2% accuracy for contact detection, which became 72.7% in [18] as the task was more complex. This task-dependency severely hinders comparison between user studies in which the performed tasks are not exactly the same; a classification measure on a specific task gives very little insight about the system's accuracy on other tasks. One should thus repeat the evaluation on every new interaction paradigm.

To overcome this shortcoming, we argue that one should consider *contact distance*, i.e., measure the actual distance between the fingertip and the contact surface. Literature explicitly mentions that location inaccuracies are a key limitation for correct gesture detection [35], with users reporting that not having sub-millimeter precision leads to recognition failures and affects user experience [34]. With this metric in mind, we now discuss existing vision systems for touch detection.

2.3.2 Hand and Fingertip Models. The seminal work of Lee et al. [38] suggests computing the parameters of a predefined hand model from visual input and inferring gestures and touches from it. The vision community has developed many hand models, and Romero et al. [57] provide a great overview of the different hand modeling approaches. In the context of HCI, most models rely on basic geometric primitives [44, 49–51, 55] or landmarks [6, 79], which allow the efficient computation of distances for touch detection. Several rigged models are controlled by shape and pose parameters [57, 65]: the shape parameters define the size of a kinematic tree, and the pose parameters define the relative angles between the phalanges. The advantage of such models is that the actual shape of the hand can be estimated in an offline calibration process and kept fixed during online hand tracking. Optimizing solely the pose parameters is known to improve tracking efficiency and accuracy [65]. For interactions where only the fingertip contact location is required, simpler models such as 2D [39] or 3D position [24], or a sphere [2, 3] are used. In our work, we propose a finger model that allows representing fingers independently, with the goal of offering a versatile approach for research on bare-handed interactions.

2.3.3 Touch Detection with Vision. Touch detection can be inferred from the estimated fingertip position or hand model using various types of cameras, including RGB [39], stereo [78], depth [17, 24, 60, 62], infrared [74], thermal [59], structured light [42], or laser-based [63] cameras. Touch and pressure estimations can also be obtained by analyzing fingertip deformation or coloration when pressing a surface [18, 19] or the skin itself [45]. Vision-based hand tracking systems, such as Google's Hand MediaPipe [79], OpenPose [6], MEgATrack [21], and UmeTrack [23], or methods using depth images [46, 48, 61, 70], can detect various touches and gestures, including pinch [23, 35, 61], swipes and taps [35, 61], drawing circles on fingers [61], grasp [73], and text entry [15, 64, 77].

However, methods evaluated solely on touch (or gesture) classification [15, 18, 19, 35, 42, 45, 59, 63, 73, 77, 78] are unlikely to provide adequate performance in other tasks. Other works, evaluating touch (or hover) distances, report errors ranging from 2-3 mm [2], 1-10 mm [62], 4.8 mm [74], 6.3 mm [64], 7 mm [28], 7.5 mm [32], 20 mm [24], or errors of 10.05 mm MSE on hand keypoints [79], 9.8 mm [21], and 9.4 mm [23] of mean-per-joint-position-error (MPJPE), or in the order of 1 cm for depth-sensor-based approaches [46, 48, 60, 61, 70]. None of these works meet the stringent requirement of sub-millimeter accuracy for touch detection explained in Section 2.1.2.

Marker-based approaches [3, 8, 67, 75, 76] have great potential, considering that tracking systems provide sub-millimeter marker precision with latency below 5 ms. However, inferring accurate touch events and touch locations from markers attached to visible areas of the hand is challenging. For example, Cerveri et al. only achieved 2-3 mm finger precision from 0.3 mm marker precision [8]. From these systems, only CIT [3] explicitly evaluates end-to-end latency and provides an estimate of touch distance accuracy. In CIT [3], a sphere fingertip model is calibrated upstream, and the author reports sub-millimeter touch distance accuracy, although the accuracy is not quantified precisely. The reported value of 25.7 ms of end-to-end latency meets the adequate performance requirements for indirect touch.

CIT only tracks a single fingertip; it is not straightforward to equip several adjacent fingertips and multiple phalanges with the proposed large rigid components, as this would be the source of collisions. Our work goes beyond CIT by allowing the unobtrusive tracking of *all* fingers and by tracking fingers' *phalanges* while maintaining adequate latency, accuracy, and precision. This notably extends the space of bare-handed interactions that can be implemented.

3 Finger Model and Tracking

3.1 Principles

Our approach builds upon the work of Bérard, who achieved fast and accurate touch detection on a single finger by modeling the surface of the fingertip as a simple sphere [3]. We replicated the calibration of the desktop plane and the use of recordings with the finger sliding and rotating while maintaining light contact with the desktop. We enhanced this approach by incorporating a kinematic model [8, 14] and by replacing the rigid body attached to the distal phalanx with four small (3 mm) independent markers directly attached to two (thumb) or three (other fingers) phalanges. This modification allows our approach to scale to the tracking of all fingers, as it eliminates the collisions that occur when attaching a rigid body to the five distal phalanges of the hand. More importantly, it enables the tracking of the phalanges, a requirement for implementing common microgestures such as tapping or sliding on another finger. In addition to modeling the fingertip surface as

a sphere, we model the phalanges of a finger as a set of two or three cylinders connected by joints. The tracking process involves matching the model's predicted marker positions (later referred to as "predictions") to the observed marker positions (later referred to as "observations"). This is done at each online frame by optimizing the *extrinsic* parameters of the model that define its *pose*: the global position and orientation of the model, and the angles of its joints. Since each finger is unique and markers are not exactly attached in the same location on each finger, accurate tracking requires that each finger has its own model adapted to its *shape* and marker placement. Before tracking, in an offline stage, we calibrate each finger's *intrinsic* parameters: cylinder lengths and radii, positions of the markers on the cylinders, and the center and radius of the contact sphere.

In the following sections, we first present our finger model, then explain how we label the observations, and detail how we optimize the extrinsic and intrinsic parameters in the "fitting" and "calibration" stages, respectively.

3.2 Finger Model

We model each finger independently, rather than as part of a global hand model, as seen in most previous approaches [21, 23, 57]. While this approach does not utilize the mechanical constraints linking the fingers together, it offers flexibility as a research tool for interaction studies. For example, experimenting with thumb-index microgestures requires only two fingers to be equipped and calibrated. This simplification benefits not only the experimenter and participant but also the system, with easier labeling and lower computational demands. Additionally, this approach is more inclusive as it accommodates users with missing fingers or polydactyly. Unless explicitly stated otherwise, we describe the tracking of a single finger model, as tracking multiple fingers involves replicating the same process.

We define a 2-phalanx model for the thumb and a 3-phalanx model for the other fingers. The 2-phalanx model has 13 and 7 degrees of freedom (DOF) in its intrinsic $\mathcal{I}$ and extrinsic $\mathcal{E}$ parameters, respectively. The 3-phalanx model has 15 and 8 DOF. Both models have 4 reflective markers attached to the fingers. The two models and their associated parameters are illustrated in Figure 2. The intrinsic parameters $\mathcal{I}$ of the 2-phalanx model are: radii (r_1, r_2) and lengths (d_2, d_4) of the 2 phalanges, longitudinal positions of markers (d_1, d_3), angles of markers $\alpha_{m1}, \alpha_{m2}, \alpha_{m3}$ ($\alpha_{m4} = 0$; in Figure 2, only α_{m1} is represented), the radius r_s of the contact sphere, and $\mathbf{s}$ (3 DOF), the 3D displacement of the contact sphere center in the distal phalanx frame defined at the inter-phalanx joint location and oriented with the distal phalanx: it defines the center $\mathbf{S}$ of the contact sphere in the world frame. The 3-phalanx model includes an additional phalanx radius r_0 and length d_0. The extrinsic parameters $\mathcal{E}$ of the 2-phalanx model are: position $\mathbf{B}$ (3 DOF) and orientation $\mathbf{R}$ (3 DOF) in the world reference frame, and the bending θ_1 of the inter-phalanx joint. The 3-phalanx model has an additional joint angle θ_0. In Appendix A, we detail how we compute all predictions $\hat{\mathbf{m}}(\mathcal{I}, \mathcal{E})$ from the intrinsic and extrinsic parameters.

Initially, we explored models with 3 markers but were unable to achieve stable tracking. As our main goal is to compute an accurate fingertip position, we bias the model towards accurate tracking of the distal phalanx by equipping it with two markers. Our marker placement has two constraints: the marker on the fingertip should be at the center of the distal part of the nail (as we assume $\alpha_{m4} = 0$), and the 4 markers should not be aligned in any pose of the finger. This setup allows us to recover the fingers' roll.

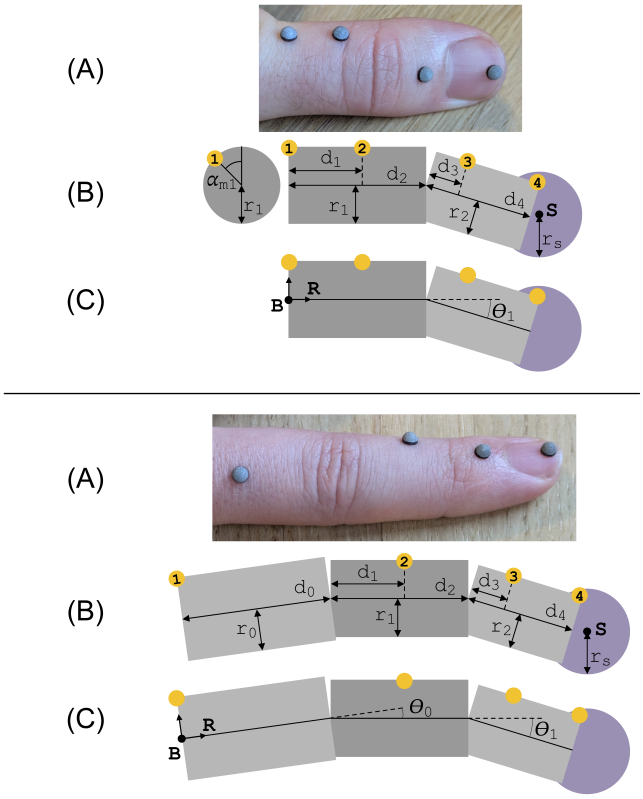

Figure 2: The 2-phalanx (top) and 3-phalanx (bottom) models: (A) Placement of the 4 markers on the fingers. (B) *Intrinsic parameters*: the 2-phalanx model has 11 parameters (13 DOF), and the 3-phalanx model has 13 parameters (15 DOF). (C) *Extrinsic parameters*: the 2-phalanx model has 3 parameters (7 DOF), and the 3-phalanx model has 4 parameters (8 DOF).

Both models feature a "contact sphere" rigidly attached to the cylinder of the distal phalanx. The contact sphere is used to accurately detect touch and release events of a fingertip and to compute the accurate touch location. Touch and release events are triggered when the distance between the contact sphere's surface and the touch surface crosses the zero threshold. The sphere model makes this distance computation trivial when the touch surface is planar (e.g. a desktop), spherical (e.g. another fingertip in a pinch gesture), or cylindrical (e.g. a phalanx in a tap or slide microgesture).

The model does not account for various physical properties of fingers, such as skin elasticity and muscle shape changes. Consequently, we cannot expect a perfect fit between observations and predictions.

3.3 Labeling

Tracking the finger model first involves labeling the observations provided by the marker tracker, i.e., associating observations with predictions. We used a simple labeling approach: at the beginning of the interaction, we asked participants to place their hands in the *initialization pose*, side by side, oriented towards the computer monitor, with palms parallel to the desktop. The leftmost and rightmost markers were assigned to the left and right hands, respectively. Then, for each hand, we labeled markers by grouping them using the known orientation of the initialization pose. After this step, the association was propagated to new frames by tracking the markers: in a new frame, we matched each observation to the closest observation in the previous frame, and the corresponding label was transferred.

If a single marker disappeared, labeling could recover as soon as a new unlabeled observation appeared: the new observation was simply associated with the prediction that lost its association. If more than one marker disappeared, tracking failed and users had to return their hands to the *initialization pose* to quickly restart it. More elaborate labeling strategies could be implemented to handle the disappearance of more than one marker, but with our high-frequency tracking of 240 Hz, markers were rarely lost. We thus settled on this simple labeling approach, which had a very low computational time (less than 20 microseconds in our experiments). Robust labeling could be achieved using machine learning [22, 58], but at the cost of a notably higher computation time (~10 ms reported by Rosskamp et al. [58]).

3.4 Fitting

The online tracking of the finger model is centered around the *Fitting* stage, illustrated in Figure 3. We consider the intrinsic parameters $\mathcal{I} = \{d_*, r_*, \alpha_{m*}\}$ to be known. Note that $\mathbf{s}$ and r_s are not used to compute the finger model poses. We optimize the extrinsic parameters $\mathcal{E}^t = \{\mathbf{R}^t, \mathbf{B}^t, \theta_*^t\}$ for each frame t. For every new frame t, we initialize the extrinsic parameters $\mathcal{E}^t$ with the result of the previous frame $\mathcal{E}^{t-1}$.

Given the labeled observations $\mathbf{m}^t = \{\mathbf{m}_i^t; i \in \{1..4\}\}$, we find $\mathcal{E}^t$ that minimizes

$$e_{fitting}(\mathbf{m}^t, \mathcal{I}; \mathcal{E}^t) = \sum_{i=1}^{4} \|\mathbf{m}_i^t - \hat{\mathbf{m}}_i^t(\mathcal{I}, \mathcal{E}^t)\|^2 \quad (1)$$

using gradient descent, where $\hat{\mathbf{m}}_i^t(\mathcal{I}, \mathcal{E}^t); i \in \{1..4\}$ are the predictions. To ease readability, we write constant arguments before the semicolon and optimized variables after. We slightly abuse notation by writing $\hat{\mathbf{m}}_i^t(\mathcal{I}, \mathcal{E}^t)$, as different markers depend only on a subset of intrinsics and extrinsics, as described in Appendix A.

To improve efficiency, we observed that $\mathbf{B}$ can be optimized more directly by not applying the gradient update to it. Instead, we update its previous position, *at each step of the gradient descent*, with the mean of the marker offsets computed in the step, i.e.,

$$\mathbf{B}_k = \mathbf{B}_{k-1} + \frac{1}{4}\sum_{i=1}^{4}\left(\mathbf{m}_i^t - \hat{\mathbf{m}}_i^t(\mathcal{I}, \mathcal{E}_{k-1}^t)\right) \quad (2)$$

where k is the index of the gradient descent step.

At 240 Hz, marker displacements are small, and the optimization usually converges in fewer than 20 steps. However, if at the end of the optimization $e_{fitting}$ is greater than 3 mm (chosen empirically), we consider the tracking to be lost and request the user to return to the *initialization pose* introduced in 3.3 to restart tracking.

3.5 Calibration

Before tracking can be used, we attach physical markers to the user's fingers and perform a calibration stage, as illustrated in Figure 4.

3.5.1 Principles. The fundamental concept is to compute user-specific intrinsic parameters $\mathcal{I}$ that match the equipped user. However, in unconstrained poses, there is an ambiguity between the intrinsic and extrinsic parameters; that is, several combinations of intrinsic and extrinsic values can result in the same marker locations. To resolve this ambiguity, we consider sequences of several frames for which the intrinsics remain identical. We attempted to optimize the full set of intrinsics in a single optimization, but convergence was not consistent, and the final finger shape appeared inaccurate. We achieved more robust and accurate calibration by splitting the process into two stages: first, calibrating the finger shape and marker positions, and second, calibrating the contact sphere.

3.5.2 Calibrating the Finger Shape. We begin by calibrating the desktop plane $\mathbf{D}$ using the protocol of CIT [3]. Next, we record the marker locations with the participants' hands in the *initialization pose* (see Section 3.3), with the additional instruction to lay the hands flat on the desktop (Figure 4, top left). This pose reduces the number of DOF to optimize for, as (1) fingers have no roll ($\mathbf{R}_0 = 0$), reducing their rotation to 2 DOF ($\mathbf{R}_{1,2}$), and (2) fingers are straight, so all joint angles are zero ($\theta_{0,1} = 0$). Note that the roll observation does not hold for the 2-phalanx model ($\mathbf{R}_0 \neq 0$). In this case, we enforce only the joint angle $\theta_1 = 0$. We record N_f "flat pose" frames, obtaining the labeled set of observations $\mathbf{m}_F = \{\mathbf{m}_i^n\}$, with $n \in \{1..N_f\}$ and $i \in \{1..4\}$. Following Eq. 1, given the set of observations $\mathbf{m}_F$, the intrinsics $\mathcal{I}$, and a sequence of "flat extrinsics" $\mathcal{E}_F = \{\mathcal{E}^n\}, n \in \{1..N_f\}$, with the fixed rotations ($\mathbf{R}_0 = 0, \theta_{0,1} = 0$), we define

$$e_{flat}(\mathbf{m}_F, \mathcal{I}, \mathcal{E}_F) = \frac{1}{N_f}\sum_{n=1}^{N_f}\sqrt{\frac{1}{4}\sum_{i=1}^{4}\|\mathbf{m}_i^n - \hat{\mathbf{m}}_i(\mathcal{I}, \mathcal{E}^n)\|^2} \quad (3)$$

In Figure 4, we use "Flat Fitting" to highlight these fixed rotations.

As the "flat pose" frames do not constrain the phalanges' length d_2, d_4, or the markers' longitudinal positions d_1, d_3, we ask the participant to perform several flexion and extension movements with their hands in mid-air (Figure 4, middle left). We record N_b frames of "bend poses", obtaining the set of labeled observations $\mathbf{m}_B = \{\mathbf{m}_\mathbf{i}^{\mathbf{N_f+n}}\}$ with $i \in 1..4$ and $n \in \{1..N_b\}$. We consider the "bend extrinsic" set $\mathcal{E}_B = \{\mathcal{E}^{N_f+n}\}, n \in \{1..N_b\}$, and define

$$e_{bend}(\mathbf{m}_B, \mathcal{I}, \mathcal{E}_B) = \frac{1}{N_b}\sum_{n=N_f+1}^{N_f+N_b}\sqrt{\frac{1}{4}\sum_{i=1}^{4}\|\mathbf{m}_i^n - \hat{\mathbf{m}}_i(\mathcal{I}, \mathcal{E}^n)\|^2} \quad (4)$$

To calibrate the intrinsic finger shape parameters $\mathcal{I} = \{d_*, r_*, \alpha_{m*}\}$, we define

$$e_{calib}(\mathcal{I}, \mathcal{E}_F, \mathcal{E}_B) = e_{flat}(\mathcal{I}, \mathcal{E}_F) + e_{bend}(\mathcal{I}, \mathcal{E}_B) \quad (5)$$

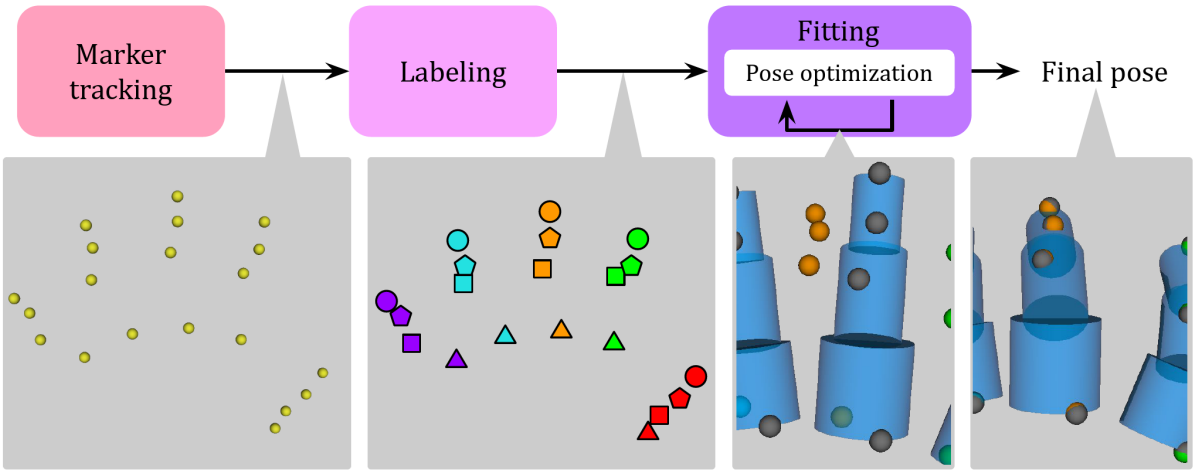

Figure 3: Tracking workflow. The marker tracker provides the observations (in yellow). In the *Labeling* stage, we associate observations with the model's markers (colors indicate fingers and shapes indicate phalanges). In the *Fitting* stage, we optimize the extrinsic parameters of the model so that the model's markers (in gray) match the labeled markers (in orange).

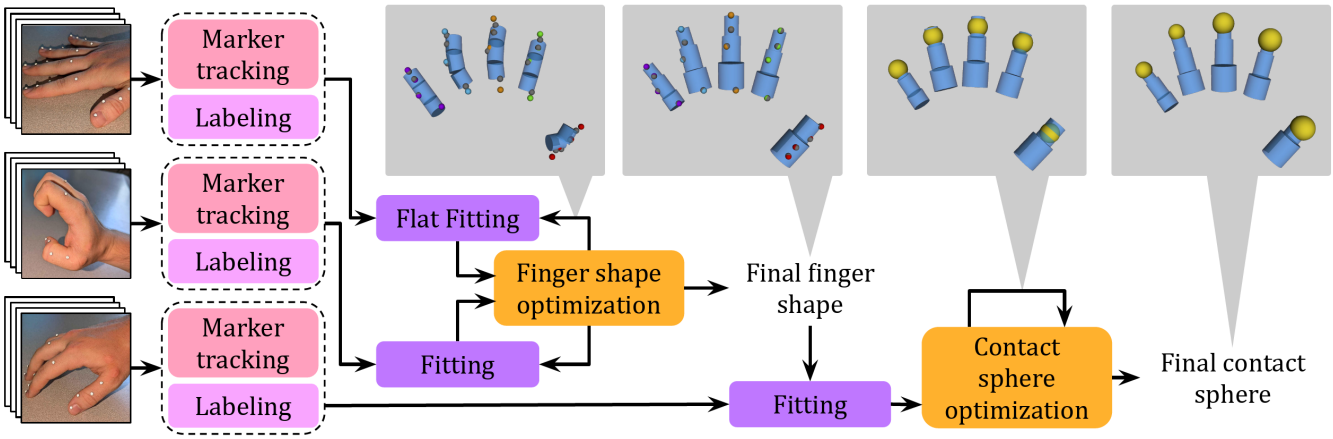

Figure 4: Calibration workflow. First, we record three hand movements (flat, bend, and touch) and label the markers. We then optimize the finger shapes using the flat and bend poses, followed by optimizing the contact spheres with the touch poses.

We start by optimizing the sets of extrinsic parameters $\mathcal{E}_F$ and $\mathcal{E}_B$ while keeping the intrinsics fixed (initialized with default mean hand size parameters), i.e., we minimize $e_{calib}(\mathcal{I}; \mathcal{E}_F, \mathcal{E}_B)$. Once optimized, we obtain a first set of extrinsics $(\mathcal{E}'_F, \mathcal{E}'_B)$, but since the intrinsics do not yet match the user's finger size, the error remains high. We then perform *a single gradient descent step* of $e_{calib}(\mathcal{E}'_F, \mathcal{E}'_B; \mathcal{I})$, i.e., we keep the extrinsics fixed and update the intrinsics. With the improved intrinsics $\mathcal{I}'$, we compute new extrinsics by re-optimizing Eq. 5 and then perform another gradient step of the intrinsics. We proceed iteratively until convergence is reached.

This optimization also includes an efficiency trick. To leverage the fact that all fingers of the "flat pose" frames lie on the desktop **D**, we could minimize an additional $e_{contact}$ to enforce the distance between the desktop and the underside of the finger to be zero. Instead, we found that we could bias the radii r_i gradient step to enforce this constraint with

$$r_i^k = r_i^{k-1} - \frac{\partial e_{calib}}{\partial r_i}(r_i^{k-1}) - \frac{c}{N_f \cdot e_{calib}} \sum_{n=1}^{N_f} d_i^n$$

where k is the index of the gradient descent step. We denote d_i^n as the signed distance between the desktop plane **D** and the underside of the finger at prediction $\hat{\mathbf{m}}_i^n$ for frame n, i.e., d_i^n is zero when the finger model touches the desktop plane **D**. We empirically set $c = 1/8$.

3.5.3 Calibrating the Contact Spheres. We extended the single-sphere CIT protocol [3] to calibrate multiple contact spheres with a single recording (Figure 4, bottom left). Users are instructed to

move their fingers on the desktop at various angles (roll, pitch, yaw) while maintaining light contact with the surface. Thus, we can assume that the distance between the fingertips' surface and the desktop is zero. We record N_t "touch pose" frames, obtaining a set of labeled observations $\mathbf{m}_T = \{\mathbf{m_i^{N_f+N_b+n}}\}$ with $i \in 1..4$ and $n \in \{1..N_t\}$.

Using the calibrated intrinsic parameters $\mathcal{I}$ from Section 3.5.2, we fit a set of extrinsic parameters $\mathcal{E}_T = \{E^{N_f+N_b+n}\}, n \in \{1..N_t\}$ to $\mathbf{m}_T$ by optimizing Eq. 1 from Section 3.4 for each frame. Note that this optimization is independent of the touch sphere parameters $(\mathbf{s}, r_s)$. Next, we optimize $(\mathbf{s}, r_s)$ by minimizing

$$e_{spheres}(\mathbf{D}, \mathcal{E}_T, \mathcal{I}; \mathbf{s}, r_s) = \sum_{n=N_f+N_b+1}^{N_f+N_b+N_t} d_{\mathrm{D}}(\mathrm{S}(\mathcal{E}^n, \mathcal{I}, \mathbf{s}), r_s)^2$$

where $\mathrm{S}(\mathcal{E}^{N_f+N_b+n}, \mathcal{I}, \mathbf{s})$ is the center of the contact sphere defined in Appendix A, and $d_{\mathrm{D}}(\mathrm{S}, r)$ defines the distance between the desktop plane $\mathbf{D}$ and the sphere with center S and radius r. This can be easily computed by subtracting the sphere radius from the projection distance between the sphere's center S and the desktop plane $\mathbf{D}$.

Since maintaining very light contact with the fingertip is challenging, participants usually press a bit harder than ideal, resulting in smaller-than-expected sphere radii. We improved the estimation of the sphere radii by interactively adjusting them and asking participants which one corresponded to their ideal touch detection. We executed this procedure only once to define a global adjustment applied to all finger radii, as it seemed to provide adequate touch detection, as evaluated in the next section. However, this procedure could be repeated for each individual finger to achieve more accurate touch detection.

4 Performance Evaluation

The marker tracking software ran on a dedicated computer connected to 10 Optitrack PrimeX 22 cameras (2048x1088 pixels set to 240 Hz). The cameras were set up approximately 1.5 m away from the center of the 1 m × 1 m × 1 m tracking volume. Marker positions were transmitted over a 1 Gbit/s network to the main computer, which executed the finger tracking and graphical interaction software programmed in C++. The main computer was equipped with a 16-core processor with a 3.4 GHz base clock (AMD Ryzen 9 5950X), an AMD Radeon Pro W7600 graphics card, and drove a 27-inch monitor operating at 240 Hz with low input lag (Corsair XENEON 27QHD240).

We evaluated our system's performance based on the following parameters identified as requiring adequate performance: end-to-end latency, precision, accuracy, and unobtrusiveness.

4.1 End-to-End Latency

We measured the fitting time using a recording of approximately 8 s (2000 frames at 240 FPS) with the demo application (Section 5), utilizing 7 fingers. On a single core, the mean time to fit one finger model was 148 μs, with 99% of fingers inferred in less than 600 μs. For the same recording, on a 16-core CPU, fitting the seven fingers in parallel took an average of 191 μs, with 99% of frames inferred in less than 600 μs. The rapid inference allowed us to refresh the graphical display at 240 Hz.

We estimated the end-to-end latency of the system using the predictive approach [7]. This required setting up our system for direct touch interaction: we positioned the computer monitor horizontally and mapped Optitrack to display coordinates. Five operators provided estimates in the range 24 ms to 25.5 ms, with an average of 24.9 ms and a standard deviation of 0.5 ms.

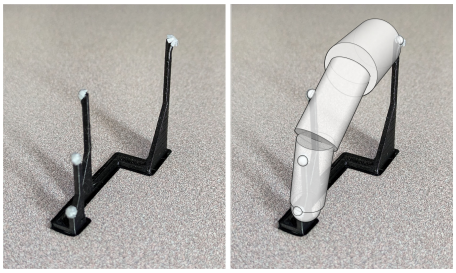

Figure 5: Rigid structure simulating a static finger to estimate the precision of our system.

	Recording 1	Recording 2	Recording 3
95th percentile	0.015	0.0095	0.0069

Table 1: Precision measurements at three different locations using a simulated static finger (see Figure 5). 95th percentile (mm) of the distances between 1000 recorded tracker outputs (4.2 s) and their center.

Participant	1	2	3	4
straight + left side	12.98	11.99	10.34	11.10
straight + centered	12.76	11.90	10.65	11.17
straight + right side	12.53	11.72	11.09	10.58
bent + left side	13.25	11.84	9.35	10.31
bent + centered	13.46	11.94	9.11	10.20
bent + right side	12.41	11.64	9.30	9.54
$(max - min)/2$	0.53	0.18	0.99	0.82

Table 2: Accuracy estimation. For each participant: distance (mm) between the center of the contact sphere and the desktop across the six finger poses shown in Figure 6, and half range of these values.

4.2 Precision

We estimated the precision of the system by 3D printing a rigid structure with four markers to simulate a static finger, as illustrated in Figure 5. We placed this structure on the desktop in three different positions and orientations, recording the system's output over 1000 frames each time. Table 1 reports the measurements for the three recordings, indicating that the precision of the system was around 0.02 mm.

Figure 6: The six finger poses used to measure touch accuracy.

4.3 Accuracy

We first conducted a touch detection sanity check using a common capacitive touch surface. We asked four external participants to perform 50 touches on a 13" iPad Pro while being tracked by our system. Since the detection rate is task-dependent (see Section 2.3.1), we chose a realistic task involving potentially subtle touches: quick reciprocal target acquisition of two easy targets (24 mm targets separated by 120 mm). Touch sphere radii were adjusted per participant (see the end of Section 3.5). The 50 acquisitions lasted between 8.24 s and 19.0 s, depending on the participant's speed. The average acquisition time per participant ranged between 0.165 s and 0.381 s per acquisition. We recorded videos showing both the iPad's screen and our system's screen, providing clear touch feedback. Analysis of these recordings reveals a 100% match in touch detection (200 touch events). We observed two instances where users moved too quickly and failed to touch the iPad. Since the non-registration of touches matched on both systems, this indicates that our system was not overly sensitive. Touch classification is not only task-dependent, as detailed in Section 2.3.1, but it also lacks sensitivity, as this empirical evidence shows.

This reinforces the motivation for measuring the accuracy of the estimated *contact distance*, i.e., the distance between the surface of the fingertip and the touch surface. As we could not find a way to obtain an accurate ground truth for this distance, we used the opposite approach: we checked the output of the tracker when the finger was at a known contact distance, i.e., at the touch threshold. We asked four participants to touch the desktop with the smallest possible force, measured the contact distance for approximately 1 s, and recorded the median. We anticipated that approximating the underside of a fingertip with a simple sphere would be one of the main sources of inaccuracy. Hence, we measured the contact distance on various parts of the fingertip by asking participants to repeat this contact in six different finger poses, as illustrated in Figure 6. For each participant, we estimated the accuracy of the tracker as the *variation* between the six averages, assuming that any global deviation could be corrected by an adjustment of the contact radius (see Section 3.5.3). Table 2 summarizes the accuracy measurements. For each participant, all measurements were within a millimeter of their center, indicating that the tracker had millimeter accuracy. This should be considered a *lower bound* estimation of the tracker's accuracy: some of the measured variability likely came from participants' difficulty in reproducing the exact same smallest possible force with various finger poses.

4.4 Unobtrusiveness

Although not directly related to the intrusiveness of the approach *during interaction*, preparing the system for tracking the ten fingers of a new participant took approximately 7 min. This process included: attaching the markers to the fingers (~3 min), recording the three sequences of hand motions (~2 min), and calibration computation time (~2 min).

During interaction, if a tracking failure occurred, it required the participant to move their hands to the default position to restart the tracking (see section 3.4). Although this action lasted only about one second, it could be quite intrusive if it occurred in the middle of a continuous gesture, such as dragging an object.

Apart from tracking failures, the system's intrusion appeared minimal during interaction. The small size of the markers (half spheres of 3 mm in diameter) reduced the risk of collisions. Due to their light weight, participants reported not feeling them. Our informal observations suggest that participants quickly forgot about the markers as their focus was on the interaction.

In the demo application discussed below, we carefully positioned the cameras of the marker tracking system to minimize the risk of tracking failures within the expected range of participants' gestures.

5 Demo Application

To illustrate the performance and utility of our approach, we developed a demonstration application using our test system. The aim was to highlight the benefits of high-performance tracking for the demonstrated interactions and to showcase the diversity of possible interactions that our approach facilitates. The goal was *not* to create the best user-centered design for the interactions, although some design principles were considered when choosing the interactions (e.g., the Kinematic Chain Model for bimanual interaction [20]). The task involved editing a 3D point cloud. We refer the reader to the accompanying video for a visual demonstration.

5.1 Overview of the Demoed Interactions

Users sit at a desk in front of a computer monitor, as illustrated in Figure 1. All interactions are indirect: user actions occur on or above the desktop, while their effects are visible in the 3D scene on the monitor. A point cloud, which is the output of scanning a physical room, is shown on the monitor. The demo allows users to select and delete points, similar to cleaning the raw output of a 3D scan. Selecting points does not immediately delete them; instead, it either adds them to or removes them from the *active set* of points. Users must explicitly trigger a "delete" command to remove the active set of points.

The transition between 2D and 3D selection modes is made automatically according to the height of the hand in relation to the desktop: simply move the hand closer to the table to switch to 2D and further away to switch to 3D.

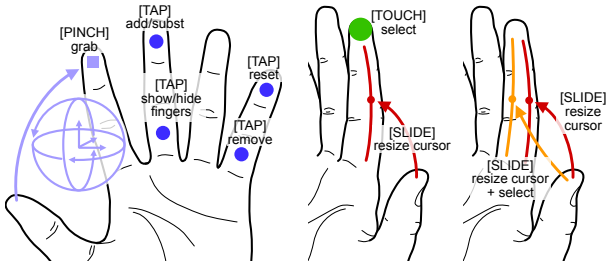

Figure 7: Gesture-command mapping in the 3D point cloud demo application. Left (non-dominant hand): (purple) Pinch with the non-dominant hand, then move and rotate the hand to move and rotate the 3D point cloud. (Blue) Tap microgestures (from either hand) on two phalanges of two fingers to specify commands. Center (dominant hand, 2D selection): (green) Touch the surface to select points, (red) Slide the thumb along the index finger to adjust the size of the 2D cursor. Right (dominant hand, 3D selection): (red) Slide the thumb along the radial side of the index finger to adjust the size of the 3D cursor, (orange) Slide the thumb along the volar side of the index finger to adjust the size of the 3D cursor while selecting points.

2D Selection: Hover-and-Touch. In 2D selection mode, hovering over the table with the index finger of the dominant hand moves a *disc* cursor across the screen. A feedforward highlights in green or red the points that will be added to or removed from the active set, depending on whether the additive or subtractive mode is active. The selected points are those hidden by the disc cursor, i.e., the 3D points whose projections on the screen lie within the boundary of the cursor. Users touch the surface to *activate* the selection, i.e., to actually add or remove the highlighted points to the active set. At this point, releasing the contact of the finger from the desktop *deactivates* the selection and switches back to the feedforward mode. Maintaining contact and sliding the finger on the surface keeps the selection active: newly highlighted points are immediately added to or removed from the active set.

3D Selection: Mid-air + Pinch. Pointing in mid-air with the *dominant* hand activates the 3D selection mode. A *spherical* cursor is shown in the 3D scene; it reproduces the displacements of the index fingertip above the desktop. A feedforward similar to that of 2D selection is activated, except the selected points are those located within the sphere of the cursor. Pinching and releasing the pinch in mid-air with the dominant hand activates and deactivates the selection, similar to touching or lifting the finger from the desktop for 2D selection.

Cursor size control: Continuous microgestures. On the dominant hand, sliding the thumb on the index finger adjusts the radii of the cursors (a disc or a sphere depending on the current selection mode) in a continuous manner. The virtual slider is mapped onto the length of the index finger on the dominant hand. In the 3D selection mode, there are actually two sliders mapped either on the side or the underside of the index finger. Both sliders control the radius of the sphere, but using the side slider only adjusts the radius, while using the underside slider also activates the selection. The use of the underside slider explains why pinching in 3D selection is possible on the entire underside of the index finger.

Other commands: Non-dominant hand. Pinching in mid-air with the *non-dominant* hand grabs the scene: the entire point cloud moves and rotates, replicating the motion of the non-dominant hand until the pinch is released. Tapping on the *middle finger distal phalanx* switches between additive and subtractive selection modes. Tapping on the *middle finger proximal phalanx* enables or disables the rendering of 3D finger models of the fingers in the 3D scene. Tapping on the *little finger distal phalanx* empties the active set. Tapping the *little finger proximal phalanx*, a difficult microgesture to execute, deletes all the points in the active set from the point cloud. All taps can be performed with the thumb of the non-dominant hand or with a finger of the dominant hand.

5.2 Implementation with Our Tracker

All of the touch, pinch, tap, and slide microgestures presented here were easily implemented by intersecting simple geometric primitives: the desktop's plane as well as the spheres and cylinders output by the tracker. Thanks to the accuracy of the tracking, misdetections were unnoticeable when fingertips touched either the desktop or other fingertips. Detecting contact between a fingertip and a phalanx relied on the radius of the phalanx, estimated during the calibration stage of the tracker. Unlike the fingertips, the phalanges' radii were not specifically optimized for touch detection. As a result, the accuracy of contact detection on phalanges was lower than that on fingertips. This was noticeable as touch was detected slightly early in both tap and pinch gestures.

In addition, the demo required a specific adaptation of the finger model of the thumb. This resulted from the fact that we do not use the same area of the thumb's fingertip surface to touch a flat surface as we do to interact with other fingers. Calibrated on the desktop, the surface of the thumb's contact sphere is located on the radial side. In thumb-to-finger contacts, the ulnar side of the thumb is used instead. We implemented an ad-hoc solution for the demo: we assumed that the thumb was symmetrical and used an additional contact sphere placed in the symmetric position from the calibrated sphere to detect contact with the other fingers. This solution probably did not provide the same accuracy as that of an actual calibration of the ulnar side, but misdetections were unnoticeable.

The demonstration application using our test system illustrates hover-and-touch interaction, mid-air interaction, and microgesture interaction. The application also illustrates the combination of these forms of interaction and bimanual interaction, whether synchronous or asynchronous. With regard to microgesture interaction, we show that the approach can implement a large number of microgestures, including tapping with any finger on any fingertip or finger phalanges. Unlike many vision systems, which need a machine learning overlay to specifically recognize a microgesture, relying on the intersection of simple geometric primitives offers great implementation flexibility. Any combination of contact between two fingers can be implemented very easily, with no new data or training required. Moreover, microgestures have mainly been studied as discrete inputs. Motivated by this observation, a very recent study by Kim et al. focuses on continuous thumb-to-finger microgestures for 3D selection in AR/VR [33]. Our demonstration of continuous

microgestures performed on two different parts of the same finger is made possible by the accuracy of the tracking. The fine control of the radii is made possible by the tracking precision.

In summary, the demonstration illustrates the benefits of high-performance tracking for these interactions, as well as the diversity of gestural interactions that the approach allows to easily experiment with.

6 Discussion

6.1 Limitations of the Approach

The main limitations of this approach stem from the line-of-sight requirement inherent in any visual tracking system. Experimenters should carefully analyze this constraint when assessing whether the presented approach is suitable for their experiment. The approach would fail to track microgestures that hide markers, such as tapping on fingernails, sliding on the upper side of a finger, or making fist gestures. Marker tracking also entails a limited volume within which markers can be tracked. This volume can be increased to some extent by moving the cameras further apart, but this comes at the cost of reduced precision and accuracy, as well as an increase in marker size. Our approach is clearly unsuitable for ecological studies on mobile interaction. All things considered, the line-of-sight requirement may be the source of intrusive tracking failures when markers are lost. This risk can be assessed by testing whether enough cameras can be set up so that markers are tracked in any hand pose required by the studied interaction. For example, if the studied interaction involves gestures made with the palm of the hand pointing upwards, some cameras should be set at the desktop's height.

6.2 System Performance

We measured the performance of our test system to have an end-to-end latency of 25 ms, a precision on the order of 0.02 mm, and a contact distance accuracy of one millimeter when touching a calibrated desk. Compared to the Quest 2's commonly used RGB machine learning approach, which achieves 11 mm accuracy according to a recent study [1], this represents an order of magnitude improvement in accuracy. A core benefit of the approach is that all interactions are built on positional tracking; that is, the approach does not use any ad-hoc process to detect touch or to measure the extent of a sliding microgesture, for example. As a consequence, the measured performances are general. For example, the measured precision and contact distance accuracy apply to any positional information, such as a pointing position in mid-air or the extent of a sliding microgesture. Similarly, detecting the touch of a fingertip or dragging objects exhibits the same low latency. Working with a uniform 3D positional input also simplifies the implementation of transitions between various forms of interactions, such as pointing on a surface and in mid-air. Thanks to the uniform performances of the system, the resulting transitions are executed in a fluid manner, as illustrated in the demo application.

The generality of positional tracking also had unexpected side benefits. Our work focused on accuracy for touch detection, but the system yielded surprisingly high precision that could be leveraged in *force* measurements. We did not explore interactions that use force (e.g., [13, 40]), but we observed that the contact distance could be used *past the touch threshold* as an estimate of the force applied to the touch gesture. A typical contact distance measured when pressing hard on the desk had a negative amplitude of typically 2 mm. With the 0.02 mm precision, the system could measure around 100 discrete levels of force. This is probably adequate to measure the number of force levels that are *controllable* by users (e.g., 10±2 reported in [13]).

Our goal was to create a system that minimizes the impact of the tracking system's imperfections when experimenting with a wide range of bare-handed interactions, i.e., to provide adequate performances. Previous work reported in Section 2.1 indicates that the latency of our system is most probably adequate for indirect interactions [3, 68]. However, our test system would probably have a measurable negative impact, even if small, on users' performances in the most demanding direct touch tasks. Concerning precision and accuracy, knowledge on adequacy levels is scarcer than for latency. Still, the performance of our system appears to be close to the most stringent precision and accuracy requirements emerging from the literature; hence, we expect that its impact on users' performance should be null or narrow at worst. Actually, thanks to its high performances, our approach could be used to *interrogate* these performance adequacy levels for various tasks, as discussed below.

6.3 Future Work

We intend to assess the adequate levels of system latency, precision, and accuracy required for common bare-hand interactions. For example, pointing or pinching in the air is a common method for text input on virtual keyboards in AR and VR (e.g., [34]). Sliding the thumb on the index finger is a promising microgesture that offers continuous control in many blind interaction contexts. Participants' performance on these tasks could be measured while artificially increasing the levels of latency, precision, and accuracy of our system. If performance degradation is observable even at optimal performance levels, this would indicate that the system does not meet adequate standards for these parameters. However, previous knowledge suggests that our system likely performs adequately for these tasks. Thus, users' performance degradation should only be observed at artificially reduced system performance levels, providing fundamental new insights into system performance requirements for these various tasks.

Modeling the underside of fingertips as spheres resulted in accurate touch detection on flat desktops but proved insufficient when the thumb pinched with another fingertip or touched a phalanx. We introduced a symmetric contact sphere in the demo application, as detailed in Section 5.2, but other fingertip surface models may be more appropriate. Exploring the use of ellipsoids, which add four degrees of freedom to the spherical model (two additional radii and a 3D orientation), seems promising. To improve the accuracy of touch detection on phalanges, we plan to study the calibration of a contact cylinder, similar to the contact sphere. This calibration could be computed from recordings of light fingertip contact on the phalanx.

7 Conclusion

We have proposed a finger model and tracking approach designed to study a wide variety of gestural interactions while minimizing

technical biases due to latency, precision, accuracy, and obtrusiveness. Our solution is based on the use of optical markers placed on the back of the fingers, geometric finger models whose placement is numerically optimized in real time, and an upstream calibration phase focusing on accurate fingertip touch detection. With our implementation of this approach, we measured performance in terms of latency, accuracy, and precision that equals or surpasses the state of the art. In particular, the accuracy of contact distance is a notable improvement compared to previous work when tracking multiple fingers. These performances should be adequate, or close to adequate, for interaction studies; their negative impact on interaction should be minimal at worst.

Acknowledgments

This work was supported by the French National Research Agency in the framework of the "France 2030" program, CONTINUUM project ANR-21-ESRE-0030 and ANR-15-IDEX-02, and ANR-11-LABX-0025-01 for PERSYVAL LabEx, as well as the project MIC ANR-22-CE33-0017.

References

[1] Diar Abdlkarim, Massimiliano Di Luca, Poppy Aves, Mohamed Maaroufi, Sang-Hoon Yeo, R. Chris Miall, Peter Holland, and Joeseph M. Galea. 2024. A methodological framework to assess the accuracy of virtual reality hand-tracking systems: A case study with the Meta Quest 2. *Behavior Research Methods* (2024), 1052–1063. doi:10.3758/s13428-022-02051-8

[2] Ankur Agarwal, Shahram Izadi, Manmohan Chandraker, and Andrew Blake. 2007. High precision multi-touch sensing on surfaces using overhead cameras. In *Second Annual IEEE International Workshop on Horizontal Interactive Human-Computer Systems (TABLETOP'07)*. IEEE, 197–200.

[3] François Bérard. 2024. Congruent Indirect Touch vs. mouse pointing performance. *International Journal of Human-Computer Studies* 187 (2024), 103261.

[4] François Bérard and Amélie Rochet-Capellan. 2012. Measuring the linear and rotational user precision in touch pointing. In *ACM International Conference on Interactive Tabletops and Surfaces (ITS)*. 183–192. doi:10.1145/2396636.2396664

[5] François Bérard, Guangyu Wang, and Jeremy R. Cooperstock. 2011. On the limits of the human motor control precision: the search for a device's human resolution. In *IFIP Conference on Human-Computer Interaction, INTERACT*. 107–122.

[6] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. 2019. Openpose: Realtime multi-person 2d pose estimation using part affinity fields. *IEEE transactions on pattern analysis and machine intelligence* 43, 1 (2019), 172–186.

[7] Elie Cattan, Amélie Rochet-Capellan, and François Bérard. 2015. A predictive approach for an end-to-end touch-latency measurement. In *Proceedings of the 2015 International Conference on Interactive Tabletops & Surfaces*. 215–218.

[8] Pietro Cerveri, Elena De Momi, N Lopomo, Gabriel Baud-Bovy, RML Barros, and Giancarlo Ferrigno. 2007. Finger kinematic modeling and real-time hand motion estimation. *Annals of biomedical engineering* 35 (2007), 1989–2002.

[9] Adrien Chaffangeon Caillet, Alix Goguey, and Laurence Nigay. 2023. μGlyph: a Microgesture Notation. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3544548.3580693

[10] Liwei Chan, Rong-Hao Liang, Ming-Chang Tsai, Kai-Yin Cheng, Chao-Huai Su, Mike Y Chen, Wen-Huang Cheng, and Bing-Yu Chen. 2013. FingerPad: private and subtle interaction using fingertips. In *Proceedings of the 26th annual ACM symposium on User interface software and technology*. 255–260.

[11] Chen Chen, Matin Yarmand, Zhuoqun Xu, Varun Singh, Yang Zhang, and Nadir Weibel. 2022. Investigating Input Modality and Task Geometry on Precision-first 3D Drawing in Virtual Reality. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 384–393. doi:10.1109/ISMAR55827.2022.00054

[12] Taizhou Chen, Tianpei Li, Xingyu Yang, and Kening Zhu. 2023. Efring: Enabling thumb-to-index-finger microgesture interaction through electric field sensing using single smart ring. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 6, 4 (2023), 1–31.

[13] Rhett Dobinson, Marc Teyssier, Jürgen Steimle, and Bruno Fruchard. 2022. MicroPress: Detecting Pressure and Hover Distance in Thumb-to-Finger Interactions. In *Proceedings of the 2022 ACM Symposium on Spatial User Interaction*. doi:10.1145/3565970.3567698

[14] Shiloh L Dockstader and A Murat Tekalp. 2002. A kinematic model for human motion and gait analysis. In *Proc. of the Workshop on Statistical Methods in Video Processing (ECCV)*. Citeseer, 49–54.

[15] John J Dudley, Keith Vertanen, and Per Ola Kristensson. 2018. Fast and precise touch-based text entry for head-mounted augmented reality with variable occlusion. *ACM Transactions on Computer-Human Interaction (TOCHI)* 25, 6 (2018), 1–40.

[16] Shariff AM Faleel, Michael Gammon, Kevin Fan, Da-Yuan Huang, Wei Li, and Pourang Irani. 2021. Hpui: Hand proximate user interfaces for one-handed interactions on head mounted displays. *IEEE Transactions on Visualization and Computer Graphics* 27, 11 (2021), 4215–4225.

[17] Neil Xu Fan and Robert Xiao. 2022. Reducing the Latency of Touch Tracking on Ad-Hoc Surfaces. *Proceedings of the ACM on Human-Computer Interaction* 6, ISS (2022), 489–499.

[18] Patrick Grady, Jeremy A Collins, Chengcheng Tang, Christopher D Twigg, Kunal Aneja, James Hays, and Charles C Kemp. 2024. Pressurevision++: Estimating fingertip pressure from diverse rgb images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 8698–8708.

[19] Patrick Grady, Chengcheng Tang, Samarth Brahmbhatt, Christopher D Twigg, Chengde Wan, James Hays, and Charles C Kemp. 2022. PressureVision: estimating hand pressure from a single RGB image. In *European Conference on Computer Vision*. Springer, 328–345.

[20] Yves Guiard. 1987. Asymmetric division of labor in human skilled bimanual action: The kinematic chain as a model. *Journal of motor behavior* 19, 4 (1987), 486–517.

[21] Shangchen Han, Beibei Liu, Randi Cabezas, Christopher D Twigg, Peizhao Zhang, Jeff Petkau, Tsz-Ho Yu, Chun-Jung Tai, Muzaffer Akbay, Zheng Wang, et al. 2020. MEgATrack: monochrome egocentric articulated hand-tracking for virtual reality. *ACM Transactions on Graphics (ToG)* 39, 4 (2020), 87–1.

[22] Shangchen Han, Beibei Liu, Robert Wang, Yuting Ye, Christopher D Twigg, and Kenrick Kin. 2018. Online optical marker-based hand tracking with deep labels. *Acm transactions on graphics (tog)* 37, 4 (2018), 1–10.

[23] Shangchen Han, Po-chen Wu, Yubo Zhang, Beibei Liu, Linguang Zhang, Zheng Wang, Weiguang Si, Peizhao Zhang, Yujun Cai, Tomas Hodan, et al. 2022. UmeTrack: Unified multi-view end-to-end hand tracking for VR. In *SIGGRAPH Asia 2022 conference papers*. 1–9.

[24] Chris Harrison, Hrvoje Benko, and Andrew D Wilson. 2011. OmniTouch: wearable multitouch interaction everywhere. In *Proceedings of the 24th annual ACM symposium on User interface software and technology*. 441–450.

[25] Christian Holz and Patrick Baudisch. 2010. The generalized perceived input point model and how to double touch accuracy by extracting fingerprints. In *ACM Conference on Human Factors in Computing Systems (CHI)*. 581–590.

[26] Benjamin F. Janzen and Robert J. Teather. 2014. Is 60 FPS Better Than 30?: The Impact of Frame Rate and Latency on Moving Target Selection. In *ACM Conference on Human Factors in Computing Systems (CHI) Extended Abstracts*. 1477–1482.

[27] Ying Jiang, Congyi Zhang, Hongbo Fu, Alberto Cannavò, Fabrizio Lamberti, Henry Y K Lau, and Wenping Wang. 2021. HandPainter - 3D Sketching in VR with Hand-based Physical Proxy. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3411764.3445302

[28] Nikhita Joshi and Daniel Vogel. 2019. An evaluation of touch input at the edge of a table. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12.

[29] Ricardo Jota, Albert Ng, Paul Dietz, and Daniel Wigdor. 2013. How fast is fast enough? a study of the effects of latency in direct-touch pointing tasks. In *Proceedings of the sigchi conference on human factors in computing systems*. 2291–2300.

[30] Hsin-Liu Kao, Christian Holz, Asta Roseway, Andres Calvo, and Chris Schmandt. 2016. DuoSkin: rapidly prototyping on-skin user interfaces using skin-friendly materials. In *Proceedings of the 2016 ACM International Symposium on Wearable Computers*. 16–23.

[31] Wolf Kienzle, Eric Whitmire, Chris Rittaler, and Hrvoje Benko. 2021. Electroring: Subtle pinch and touch detection with a ring. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–12.

[32] David Kim, Shahram Izadi, Jakub Dostal, Christoph Rhemann, Cem Keskin, Christopher Zach, Jamie Shotton, Timothy Large, Steven Bathiche, Matthias Niessner, et al. 2014. Retrodepth: 3d silhouette sensing for high-precision input on and above physical surfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1377–1386.

[33] Jina Kim, Yang Zhang, and Sang Ho Yoon. 2025. T2IRay: Design of Thumb-to-Index based Indirect Pointing for Continuous and Robust AR/VR Input. *To appear in the Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (2025).

[34] Taejun Kim, Amy Karlson, Aakar Gupta, Tovi Grossman, Jason Wu, Parastoo Abtahi, Christopher Collins, Michael Glueck, and Hemant Bhaskar Surale. 2023. Star: Smartphone-analogous typing in augmented reality. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–13.

[35] Kenrick Kin, Chengde Wan, Ken Koh, Andrei Marin, Necati Cihan Camgöz, Yubo Zhang, Yujun Cai, Fedor Kovalev, Moshe Ben-Zacharia, Shannon Hoople, et al. 2024. STMG: A Machine Learning Microgesture Recognition System for Supporting Thumb-Based VR/AR Input. In *Proceedings of the 2024 CHI Conference*

on Human Factors in Computing Systems. 1–15.

[36] Yuki Kubo, Yuto Koguchi, Buntarou Shizuki, Shin Takahashi, and Otmar Hilliges. 2019. Audiotouch: Minimally invasive sensing of micro-gestures via active bio-acoustic sensing. In *Proceedings of the 21st international conference on human-computer interaction with mobile devices and services*. 1–13.

[37] DoYoung Lee, SooHwan Lee, and Ian Oakley. 2020. Nailz: Sensing hand input with touch sensitive nails. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–13.

[38] Jintae Lee and Tosiyasu L Kunii. 1995. Model-based analysis of hand posture. *IEEE Computer Graphics and applications* 15, 5 (1995), 77–86.

[39] Lik Hang Lee, Tristan Braud, Farshid Hassani Bijarbooneh, and Pan Hui. 2019. TiPoint: Detecting fingertip for mid-air interaction on computational resource constrained smartglasses. In *Proceedings of the 2019 ACM International Symposium on Wearable Computers*. 118–122.

[40] Lik Hang Lee, Kit Yung Lam, Tong Li, Tristan Braud, Xiang Su, and Pan Hui. 2019. Quadmetric optimized thumb-to-finger interaction for force assisted one-handed text entry on mobile headsets. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 3, 3 (2019), 1–27.

[41] Darren Leigh, Clifton Forlines, Ricardo Jota, Steven Sanders, and Daniel Wigdor. 2014. High Rate, Low-latency Multi-touch Sensing with Simultaneous Orthogonal Multiplexing. In *ACM Symposium on User Interface Software and Technology (UIST)*. 355–364. doi:10.1145/2642918.2647353

[42] Chen Liang, Xutong Wang, Zisu Li, Chi Hsia, Mingming Fan, Chun Yu, and Yuanchun Shi. 2023. Shadowtouch: Enabling free-form touch-based hand-to-surface interaction with wrist-mounted illuminant by shadow projection. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–14.

[43] I Scott MacKenzie and Colin Ware. 1993. Lag as a determinant of human performance in interactive systems. In *Proceedings of the INTERACT'93 and CHI'93 conference on Human factors in computing systems*. 488–493.

[44] Stan Melax, Leonid Keselman, and Sterling Orsten. 2013. Dynamics based 3D skeletal hand tracking. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. 184–184.

[45] Vimal Mollyn and Chris Harrison. 2024. EgoTouch: On-Body Touch Input Using AR/VR Headset Cameras. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–11.

[46] Franziska Mueller, Micah Davis, Florian Bernard, Oleksandr Sotnychenko, Mickeal Verschoor, Miguel A Otaduy, Dan Casas, and Christian Theobalt. 2019. Real-time pose and shape reconstruction of two interacting hands with a single depth camera. *ACM Transactions on Graphics (ToG)* 38, 4 (2019), 1–13.

[47] Albert Ng, Michelle Annett, Paul Dietz, Anoop Gupta, and Walter F Bischof. 2014. In the blink of an eye: Investigating latency perception during stylus interaction. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1103–1112.

[48] Markus Oberweger, Paul Wohlhart, and Vincent Lepetit. 2019. Generalized feedback loop for joint hand-object pose estimation. *IEEE transactions on pattern analysis and machine intelligence* 42, 8 (2019), 1898–1912.

[49] Iasonas Oikonomidis, Nikolaos Kyriazis, and Antonis A Argyros. 2012. Tracking the articulated motion of two strongly interacting hands. In *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 1862–1869.

[50] Iason Oikonomidis, Nikolaos Kyriazis, Antonis A Argyros, et al. 2011. Efficient model-based 3D tracking of hand articulations using Kinect.. In *BmVC*, Vol. 1. 3.

[51] Iason Oikonomidis, Manolis IA Lourakis, and Antonis A Argyros. 2014. Evolutionary quasi-random search for hand articulations tracking. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 3422–3429.

[52] Andriy Pavlovych and Carl Gutwin. 2012. Assessing target acquisition and tracking performance for complex moving targets in the presence of latency and jitter. In *Proceedings of the 2012 Graphics Interace Conference*. 109–116.

[53] Andriy Pavlovych and Wolfgang Stuerzlinger. 2009. The tradeoff between spatial jitter and latency in pointing tasks. In *ACM SIGCHI symposium on Engineering Interactive Computing Systems (EICS)*. 187–196. doi:10.1145/1570433.1570469

[54] Manuel Prätorius, Dimitar Valkov, Ulrich Burgbacher, and Klaus Hinrichs. 2014. DigiTap: an eyes-free VR/AR symbolic input device. In *Proceedings of the 20th ACM Symposium on Virtual Reality Software and Technology*. 9–18.

[55] James M Rehg and Takeo Kanade. 1994. Visual tracking of high dof articulated structures: an application to human hand tracking. In *Computer Vision—ECCV'94: Third European Conference on Computer Vision Stockholm, Sweden, May 2–6 1994 Proceedings, Volume II 3*. Springer, 35–46.

[56] Mark Richardson, Matt Durasoff, and Robert Wang. 2020. Decoding surface touch typing from hand-tracking. In *Proceedings of the 33rd annual ACM symposium on user interface software and technology*. 686–696.

[57] Javier Romero, Dimitrios Tzionas, and Michael J. Black. 2017. Embodied Hands: Modeling and Capturing Hands and Bodies Together. *ACM Transactions on Graphics, (Proc. SIGGRAPH Asia)* 36, 6 (Nov. 2017).

[58] Janis Rosskamp, Rene Weller, Thorsten Kluss, Jaime L Maldonado C, and Gabriel Zachmann. 2020. Improved CNN-based marker labeling for optical hand tracking. In *Virtual Reality and Augmented Reality: 17th EuroVR International Conference, EuroVR 2020, Valencia, Spain, November 25–27, 2020, Proceedings 17*. Springer, 165–177.

[59] Elliot N Saba, Eric C Larson, and Shwetak N Patel. 2012. Dante vision: In-air and touch gesture sensing for natural surface interaction with combined depth and thermal cameras. In *2012 IEEE International Conference on Emerging Signal Processing Applications*. IEEE, 167–170.

[60] Vivian Shen, James Spann, and Chris Harrison. 2021. Farout touch: Extending the range of ad hoc touch sensing with depth cameras. In *Proceedings of the 2021 ACM Symposium on Spatial User Interaction*. 1–12.

[61] Mohamed Soliman, Franziska Mueller, Lena Hegemann, Joan Sol Roo, Christian Theobalt, and Jürgen Steimle. 2018. Fingerinput: Capturing expressive single-hand thumb-to-finger microgestures. In *Proceedings of the 2018 ACM International Conference on Interactive Surfaces and Spaces*. 177–187.

[62] Srinath Sridhar, Anders Markussen, Antti Oulasvirta, Christian Theobalt, and Sebastian Boring. 2017. Watchsense: On-and above-skin input sensing through a wearable depth sensor. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 3891–3902.

[63] Paul Streli, Jiaxi Jiang, Juliete Rossie, and Christian Holz. 2023. Structured Light Speckle: Joint Ego-Centric Depth Estimation and Low-Latency Contact Detection via Remote Vibrometry. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–12.

[64] Paul Streli, Mark Richardson, Fadi Botros, Shugao Ma, Robert Wang, and Christian Holz. 2024. TouchInsight: Uncertainty-aware Rapid Touch and Text Input for Mixed Reality from Egocentric Vision. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–16.

[65] David Joseph Tan, Thomas Cashman, Jonathan Taylor, Andrew Fitzgibbon, Daniel Tarlow, Sameh Khamis, Shahram Izadi, and Jamie Shotton. 2016. Fits like a glove: Rapid and reliable hand shape personalization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 5610–5619.

[66] Robert J. Teather, Andriy Pavlovych, Wolfgang Stuerzlinger, and I. Scott MacKenzie. 2009. Effects of tracking technology, latency, and spatial jitter on object movement. In *IEEE Symposium on 3D User Interfaces (3DUI)*. 43–50. doi:10.1109/3DUI.2009.4811204

[67] Hsin-Ruey Tsai, Te-Yen Wu, Da-Yuan Huang, Min-Chieh Hsiu, Jui-Chun Hsiao, Yi-Ping Hung, Mike Y Chen, and Bing-Yu Chen. 2017. Segtouch: Enhancing touch input while providing touch gestures on screens using thumb-to-index-finger gestures. In *Proceedings of the 2017 CHI Conference Extended Abstracts on Human Factors in Computing Systems*. 2164–2171.

[68] Daniel Vogel and Ravin Balakrishnan. 2005. Distant freehand pointing and clicking on very large, high resolution displays. In *ACM Symposium on User Interface Software and Technology (UIST)*. 33–42.

[69] Anandghan Waghmare, Youssef Ben Taleb, Ishan Chatterjee, Arjun Narendra, and Shwetak Patel. 2023. Z-Ring: Single-Point Bio-Impedance Sensing for Gesture, Touch, Object and User Recognition. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3544548.3581422

[70] Chengde Wan, Thomas Probst, Luc Van Gool, and Angela Yao. 2019. Self-supervised 3d hand pose estimation through training by fitting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10853–10862.

[71] Eric Whitmire, Mohit Jain, Divye Jain, Greg Nelson, Ravi Karkar, Shwetak Patel, and Mayank Goel. 2017. Digitouch: Reconfigurable thumb-to-finger input and text entry on head-mounted displays. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 1, 3 (2017), 1–21.

[72] Daniel Wigdor, Clifton Forlines, Patrick Baudisch, John Barnwell, and Chia Shen. 2007. Lucid touch: a see-through mobile device. In *ACM Symposium on User Interface Software and Technology (UIST)*. 269–278. doi:10.1145/1294211.1294259

[73] Erwin Wu, Ye Yuan, Hui-Shyong Yeo, Aaron Quigley, Hideki Koike, and Kris M Kitani. 2020. Back-hand-pose: 3d hand pose estimation for a wrist-worn camera via dorsum deformation network. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. 1147–1160.

[74] Robert Xiao, Scott Hudson, and Chris Harrison. 2016. Direct: Making touch tracking on ordinary surfaces practical with hybrid depth-infrared sensing. In *Proceedings of the 2016 ACM international conference on interactive surfaces and spaces*. 85–94.

[75] Zheer Xu, Weihao Chen, Dongyang Zhao, Jiehui Luo, Te-Yen Wu, Jun Gong, Sicheng Yin, Jialun Zhai, and Xing-Dong Yang. 2020. Bitiptext: Bimanual eyes-free text entry on a fingertip keyboard. In *Proceedings of the 2020 CHI conference on human factors in computing systems*. 1–13.

[76] Zheer Xu, Pui Chung Wong, Jun Gong, Te-Yen Wu, Aditya Shekhar Nittala, Xiaojun Bi, Jürgen Steimle, Hongbo Fu, Kening Zhu, and Xing-Dong Yang. 2019. Tiptext: Eyes-free text entry on a fingertip keyboard. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*. 883–899.

[77] Xin Yi, Chun Yu, Mingrui Zhang, Sida Gao, Ke Sun, and Yuanchun Shi. 2015. Atk: Enabling ten-finger freehand typing in air based on 3d hand tracking data. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*. 539–548.

[78] Chun Yu, Xiaoying Wei, Shubh Vachher, Yue Qin, Chen Liang, Yueting Weng, Yizheng Gu, and Yuanchun Shi. 2019. Handsee: enabling full hand interaction on smartphone with front camera-based stereo vision. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–13.

[79] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214* (2020).
[80] Yang Zhang, Wolf Kienzle, Yanjun Ma, Shiu S. Ng, Hrvoje Benko, and Chris Harrison. 2019. ActiTouch: Robust Touch Detection for On-Skin AR/VR Interfaces. In *ACM Symposium on User Interface Software and Technology (UIST)*. 1151–1159. doi:10.1145/3332165.3347869
[81] Martina Ziefle. 1998. Effects of Display Resolution on Visual Performance. *Human Factors* (1998), 554–568. doi:10.1518/001872098779649355

A Appendix: Computation of Markers and Contact Sphere Positions

Given the finger intrinsic $\mathcal{I} = \{d_0, d_1, d_2, d_3, d_4; r_0, r_1, r_2; \alpha_{m1}, \alpha_{m2}, \alpha_{m3}; \mathbf{s}, r_s\}$ and extrinsic $\mathcal{E} = (\mathbf{B}; \mathbf{R}; \theta_0, \theta_1)$ parameters, the following equations compute the positions of the finger model markers ($\hat{\mathbf{m}}_{\mathbf{1..4}}$) and contact sphere center ($\mathbf{S}$) in the world reference frame. We make explicit which parameters affect each marker location.

To improve readability we omit the multiplication with the inverse quaternion in the rotations. I.e. we write $q(\theta) \cdot \mathbf{v}$ in place of $q(\theta) \cdot \mathbf{v} \cdot q(\theta)^{-1}$.

A.1 Two Phalanges Model

$$\hat{\mathbf{m}}_\mathbf{1}(\mathbf{R}, \mathbf{B}, \alpha_{m1}, r_1) = q(\mathbf{R}) \begin{pmatrix} 0 \\ cos(\alpha_{m1}) \cdot r_1 \\ sin(\alpha_{m1}) \cdot r_1 \end{pmatrix} + \mathbf{B}$$

$$\hat{\mathbf{m}}_\mathbf{2}(\mathbf{R}, \mathbf{B}, \alpha_{m2}, d_1, r_1) = q(\mathbf{R}) \begin{pmatrix} d_1 \\ cos(\alpha_{m2}) \cdot r_1 \\ sin(\alpha_{m2}) \cdot r_1 \end{pmatrix} + \mathbf{B}$$

$$\hat{\mathbf{m}}_\mathbf{3}(\mathbf{R}, \mathbf{B}, \alpha_{m3}, d_2, d_3, r_2, \theta_1) = q(\mathbf{R}) \left[q(\theta_1) \begin{pmatrix} d_3 \\ cos(\alpha_{m3}) \cdot r_2 \\ sin(\alpha_{m3}) \cdot r_2 \end{pmatrix} + \begin{pmatrix} d_2 \\ 0 \\ 0 \end{pmatrix} \right] + \mathbf{B}$$

$$\hat{\mathbf{m}}_\mathbf{4}(\mathbf{R}, \mathbf{B}, d_2, d_4, r_2, \theta_1) = q(\mathbf{R}) \left[q(\theta_1) \begin{pmatrix} d_4 \\ r_2 \\ 0 \end{pmatrix} + \begin{pmatrix} d_2 \\ 0 \\ 0 \end{pmatrix} \right] + \mathbf{B}$$

$$\mathbf{S}(\mathbf{R}, \mathbf{B}, d_2, \theta_1, \mathbf{s}) = q(\mathbf{R}) \left[q(\theta_1) \cdot \mathbf{s} + \begin{pmatrix} d_2 \\ 0 \\ 0 \end{pmatrix} \right] + \mathbf{B}$$

A.2 Three Phalanges Model

$$\hat{\mathbf{m}}_1(\mathbf{R}, \mathbf{B}, \alpha_{m1}, r_0) = q(\mathbf{R}) \begin{pmatrix} 0 \\ cos(\alpha_{m1}) \cdot r_0 \\ sin(\alpha_{m1}) \cdot r_0 \end{pmatrix} + \mathbf{B}$$

$$\hat{\mathbf{m}}_2(\mathbf{R}, \mathbf{B}, \theta_0, \alpha_{m2}, d_0, d_1, r_1) = q(\mathbf{R}) \left[q(\theta_0) \begin{pmatrix} d_1 \\ cos(\alpha_{m2}) \cdot r_1 \\ sin(\alpha_{m2}) \cdot r_1 \end{pmatrix} + \begin{pmatrix} d_0 \\ 0 \\ 0 \end{pmatrix} \right] + \mathbf{B}$$

$$\hat{\mathbf{m}}_3(\mathbf{R}, \mathbf{B}, \theta_0, \theta_1, \alpha_{m3}, d_0, d_2, d_3, r_2) = q(\mathbf{R}) \left(q(\theta_0) \left[q(\theta_1) \begin{pmatrix} d_3 \\ cos(\alpha_{m3}) \cdot r_2 \\ sin(\alpha_{m3}) \cdot r_2 \end{pmatrix} + \begin{pmatrix} d_2 \\ 0 \\ 0 \end{pmatrix} \right] + \begin{pmatrix} d_0 \\ 0 \\ 0 \end{pmatrix} \right) + \mathbf{B}$$

$$\hat{\mathbf{m}}_4(\mathbf{R}, \mathbf{B}, \theta_0, \theta_1, d_0, d_2, d_4, r_2) = q(\mathbf{R}) \left(q(\theta_0) \left[q(\theta_1) \begin{pmatrix} d_4 \\ r_2 \\ 0 \end{pmatrix} + \begin{pmatrix} d_2 \\ 0 \\ 0 \end{pmatrix} \right] + \begin{pmatrix} d_0 \\ 0 \\ 0 \end{pmatrix} \right) + \mathbf{B}$$

$$\mathbf{S}(\mathbf{R}, \mathbf{B}, d_0, d_2, \theta_0, \theta_1, \mathbf{s}) = q(\mathbf{R}) \left(q(\theta_1) \left[q(\theta_2) \cdot \mathbf{s} + \begin{pmatrix} d_2 \\ 0 \\ 0 \end{pmatrix} \right] + \begin{pmatrix} d_0 \\ 0 \\ 0 \end{pmatrix} \right) + \mathbf{B}$$